

CSE 4501: Operating Systems Lecture

Belady's Anomaly, Segmentation & Paging

February 9, 2026

1 Introduction to Belady's Anomaly

In page replacement algorithms, it is generally expected that increasing the number of page frames results in fewer page faults. However, some algorithms, such as **FIFO (First-In-First-Out)**, can exhibit **Belady's Anomaly**, where the page fault rate actually *increases* when more frames are added.

1.1 Theoretical Context: The Stack Property

Algorithms that do not exhibit Belady's Anomaly (like LRU or Optimal) follow the **Stack Property**.

Definition: Stack Property

For any reference string S , the set of pages in memory for n frames must be a subset of the pages in memory for m frames, where $m > n$ at any time t .

$$M(n, t) \subseteq M(m, t)$$

1.2 Analysis of FIFO (Reference String: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5)

1.2.1 FIFO with 3 Frames

Access	1	2	3	4	1	2	5	1	2	3	4	5
F1	1	1	1	4	4	4	5	5	5	5	5	5
F2		2	2	2	1	1	1	1	1	3	3	3
F3			3	3	3	2	2	2	2	2	4	4
Fault?	Y	Y	Y	Y	Y	Y	Y			Y	Y	

Total Faults: 9

1.2.2 FIFO with 4 Frames

Access	1	2	3	4	1	2	5	1	2	3	4	5
F1	1	1	1	1	1	1	5	5	5	5	4	4
F2		2	2	2	2	2	2	1	1	1	1	5
F3			3	3	3	3	3	3	2	2	2	2
F4				4	4	4	4	4	4	3	3	3
Fault?	Y	Y	Y	Y			Y	Y	Y	Y	Y	Y

Total Faults: 10 (The Anomaly!)

1.3 Proof: Why FIFO Fails the Stack Property

To prove that FIFO is not a stack algorithm, we check **Step 7** (Page Access: 5):

- **Memory with 3 Frames (M_3):** $\{5, 1, 2\}$
- **Memory with 4 Frames (M_4):** $\{5, 2, 3, 4\}$
- **Result:** $\{5, 1, 2\} \not\subseteq \{5, 2, 3, 4\}$. Page **1** is missing from the larger set.

Homework: LRU Analysis Task

Lab Task

Using the same reference string: **1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5**: Simulate LRU for **3 and 4 frames** and prove the **Stack Property** holds.

2 Segmentation Address Translation Practice

Consider a system using segmentation with the following Segment Table:

Segment #	Base Address	Length (Limit)
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

2.1 Problem A: Basic Translation

Compute the physical addresses for the following logical addresses (s, d) :

- a) (0, 430)
- b) (1, 11)
- c) (2, 120)
- d) (3, 500)
- e) (4, 95)

2.2 Problem B: Segment Limits

Identify which of the following logical addresses will result in a *segmentation fault*:

- (1, 15)
- (4, 100)
- (0, 599)

3 Two-Level Memory Management: Segmentation with Paging

In some architectures (like x86 in protected mode), a logical address is divided into: **(Segment Number s , Page Number p , Offset d)**.

- The Segment Table points to the base of each segment's Page Table.
- Each Page Table Entry (PTE) contains the Frame Number in physical memory.
- **Physical Address** = (Frame Number $\times$ Page Size) + Offset.

Assumptions

Page size = 512 bytes $\implies$ offset d is 9 bits ($2^9 = 512$).

3.1 Task 1: Basic Translation

Segment Table:

Segment #	PT Base Address	Length (in pages)
0	100	4
1	200	3
2	300	2

Page Tables (Frame Mappings):

- **Seg 0:** P0→F5, P1→F8, P2→F2, P3→F7
- **Seg 1:** P0→F10, P1→F3, P2→F9
- **Seg 2:** P0→F6, P1→F4

Convert logical addresses (s, p, d) to physical addresses: **a) (0, 2, 100), b) (1, 3, 200), c) (2, 1, 500), d) (0, 1, 300), e) (3, 0, 100).**

3.2 Task 2: Error Detection

Determine if each address is valid; if yes, compute physical address: **a) (0, 1, 511), b) (1, 2, 0), c) (0, 2, 100), d) (1, 1, 600), e) (0, 0, 512).**